

Deep Learning for Visual Computing - Assignment 1.

Anna Laczkó (12347556)
Marcell Szekrényes (12347300)

May 13, 2025

1 Loss Function and Cross-Entropy

The loss function calculates the difference between the predicted outputs of the model and the true target values. With this value we can optimize our training while running. This is also a measure of the model's performance.

2 Cross-Entropy

Cross-entropy measures the dissimilarity between the predicted class probabilities. Penalizes incorrect confidence predictions more than correct confidence predictions.

Requirements for cross-entropy loss:

- Labels must be one-hot encoded or be in an integer form
- Predicted class scores must be log-probabilities or raw logits

2.1 Training, Validation, and Test Sets

- **Training set:** Used to update the model parameters.
- **Validation set:** Used to monitor performance during training and tune hyperparameters without overfitting.
- **Test set:** Used only once for final evaluation, to estimate general performance.

Using all of them ensures that the model performs well not just on seen data, but also on unseen, new data.

3 Regularization Techniques

When training on a dataset we usually witness overfitting. To battle this, we use several regularization techniques that can improve model performance, make the model generalize better or make it more robust.

3.1 Implementation

We used the following two regularization in different variations.

3.1.1 Data Augmentation:

For Data Augmentation we followed the task description quite closely after realising how long it takes to run the models. We chose random cropping, and horizontal mirroring first then added random rotation.

So in the end we had 3 variations, which we named in WandDB the following way:

- No Augmentation: `<runname>_NA_<weight_decay>`

- Random Crop & Horizontal Mirror: <runname>_RCHM_<weight_decay>
- Random Crop & Horizontal Mirror & Rotation: <runname>_RCHMR_<weight_decay>

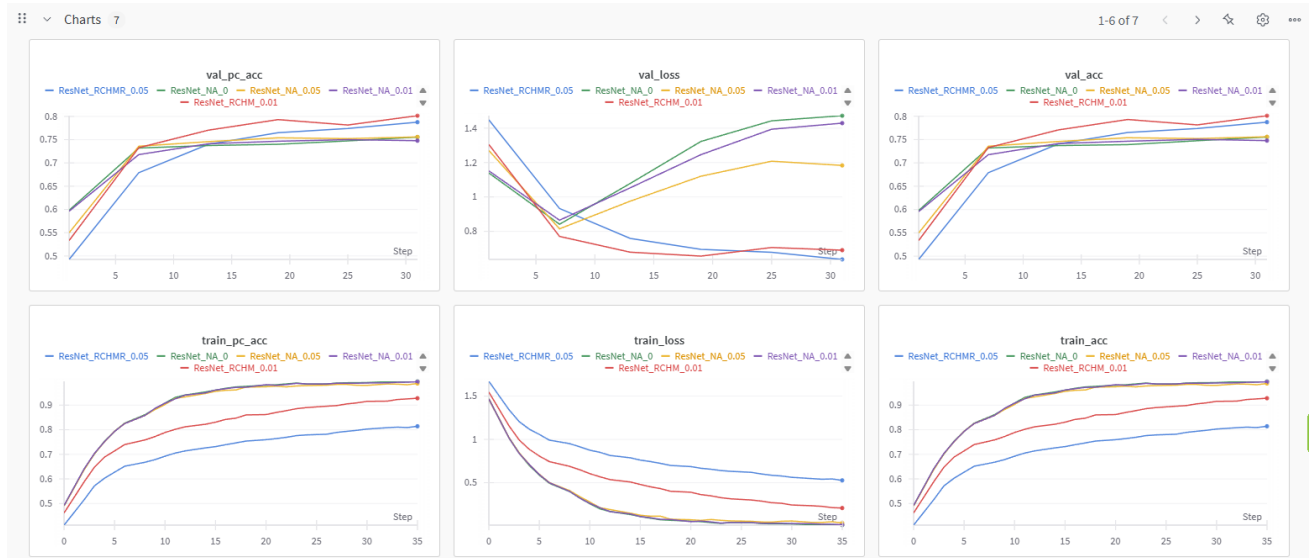
3.1.2 Weight Decay:

Used the default weight_decay, which is 0.01, then chose the following two other values: 0.05 and 0. The naming is as follows:

- No decay: <runname>_<augmentation>_0>
- 0.01: <runname>_<augmentation>_0.01>
- 0.05: <runname>_<augmentation>_0.05

When we analysed the plot we saw that the validation loss gets a bit better when choosing a bigger value for weight decay. We can say that the weight decay helps with the overfit, but not nearly as much than the data augmentation.

3.1.3 Runs in WanDB and tabular results



Model	Train PerClassAcc	Val PerClassAcc	Test PerClassAcc
ResNet_NA_0	0.9967	0.7555	0.7505
ResNet_NA_0.01	0.9951	0.7477	0.7530
ResNet_NA_0.05	0.9880	0.7559	0.7453
ResNet_RCHM_0.01	0.9281	0.8016	0.7967
ResNet_RCHMR_0.05	0.8140	0.7876	0.7794

The results indicate that while regularization and data augmentation help reduce overfitting and improve generalization, there appears to be a trade-off. Moderate regularization achieves the highest validation and test accuracy, while stronger regularization or augmentation slightly reduces performance on unseen data. Models with very high training accuracy tend to show lower validation/test accuracy, supporting the idea of overfitting. Overall, a balanced amount of regularization seems most effective for generalization.

We will use the RCHM_0.01 combination for the other models as well.

4 CNN vs. Vision Transformer (ViT)

4.1 CNNs

CNNs process 2D data (images) to detect local patterns on them. They do this by sliding a filter over the image to create feature maps to highlight important details. In the different layers the network

builds up a hierarchy for the features from edges, to complex patterns.

4.2 ViTs

ViTs process 2D data, by handling them as a sequence of 1D vectors. ViTs split the image into equal sized patches, flatten them and turn each of them into a vector. These vectors then passed through a transformer encoder, where an attention mechanism helps detect important details. For preserving spatial information ViTs add a positional embedding.

4.3 ViT Implementation

The ViT model was adapted from this Article. This article is about how to build up a ViT from scratch and we found it really useful to be able to do it step by step. Also it was already almost compatible with our code, so it spared a lot of debugging for us.

Patch Embedding (Lines 15–28)

The `patchify()` function splits each image into patches, flattens them, and projects them into a hidden dimension using a linear layer. This converts images into input sequences suitable for a transformer, that we will use later.

Positional Embedding(Lines 30–35)

Since transformers won't be able to see the spatial arrangement, we add positional encodings (`get_positional_embeddings`). These are added to the patch embeddings in the forward pass.

Multi-head Self-Attention (Lines 37–70)

The `MyMSA` class implements Multi-Head Self-Attention. Its goal is to allow the model to attend to different parts of the input sequence simultaneously, independently using multiple "attention heads". For each head we generate queries, keys and values which then used to compute scores to capture relationships between the patch vectors. The outputs in the end are all combined so the model can understand the different parts of the patches at the same time

Transformer Encoder Block (Lines 72–90)

This block combines the MHSA and the Multi-Layer Perceptron to process the input. It will first use MHSA to capture the relationships, then it will refine those with MLP.

Vision Transformer (Lines 92–150)

The `MyViT` class implements a Vision Transformer (ViT) model. It takes an input image with shape (C, H, W) , divides it into patches, and maps them using a linear layer.

The `forward` method performs these operations: it tokenizes the patches, adds the classification token and positional embeddings, and puts the tokens into the transformer blocks. Finally, the output related to the classification token is used to predict the class label through the MLP.

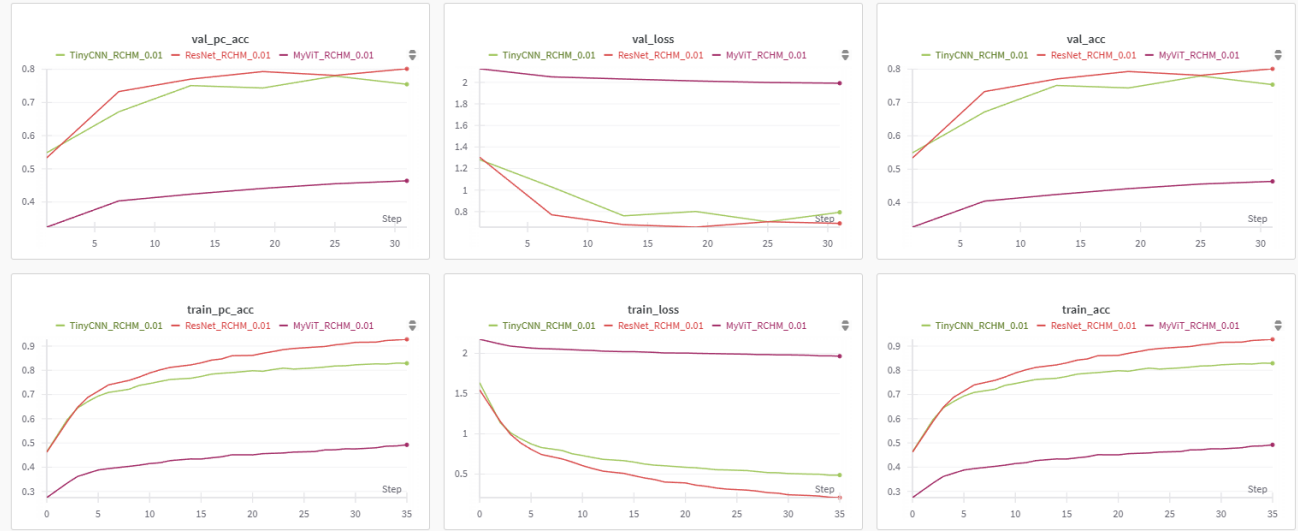
5 Comparison

After optimizing ResNet18 we used the same regularization for our two implemented models. As we expected, these performed with a lower accuracy, which we collected in this table:

Model	Train PerClassAcc	Val PerClassAcc	Test PerClassAcc
ResNet_RCHM_0.01	0.9281	0.8016	0.7967
TinyCNN_RCHM_0.01	0.8289	0.7548	0.7690
MyViT_RCHM_0.01	0.4924	0.4638	0.4635

We can see that the difference between the best validation accuracy and the final test accuracy is really small, almost nothing. We would like to also add that after the last validation we still had some training cycles which could mean that the difference between the validation accuracy and the test accuracy got smaller. We noticed this issue late in the end and we had no time to correct this slight mistake.

Also, the similarity between the validation and test accuracy also means that we probably don't have data leakage from the training set, and our model can consistently perform the same way on unknown data.



6 Optimizer & Learning Rate scheduler

- **Optimizer:** AdamW
 - Learning rate: 0.001
 - `amsgrad = True`
- **Learning Rate Scheduler:** ExponentialLR
 - Gamma (decay factor): 0.9
- **Loss Function:** CrossEntropyLoss

7 Conclusion & Final Thoughts

What we can really notice is that the ViT is considerably worse than the others. The explanation for this is simple: the ViT has to learn all the underlying structure and takes a lot more data (and time) to be trained on a similar level like CNNs. So we can expect better results, even the best result, if we have a lot more time, a lot more data (and a bigger, stronger GPU).

We were surprised with our CNN too, but for the opposite reason: we didn't expect such good results compared to ResNet.

Also interesting "note to self": we first forgot to set ResNet to be untrained, which of course also resulted in much better results. It was interesting to see how much difference can be achieved with pre-trained weights.